Mini CRM distribuído

Documentação de arquitetura (ARC42) — Projeto PJBL, Arquitetura de Software

Alunos	Pedro Henrique Z. Costa · Lincoln Neto · Nicolas Tancon · Leandro Canha
Versão	1.0 — novembro de 2026
Repositório central	github.com/Le3Dev/minicrm-arquitetura
Aplicação no ar	shell-zeta-puce.vercel.app
Docker Hub	hub.docker.com/u/leandrocanha
Vídeo (YouTube, não listado)	https://youtu.be/f_G2twBHaKU

Nos diagramas e tabelas, **azul** indica componentes da dupla A e **laranja** os do aluno B.

Sumário

1	Introdução e metas	7	Visão de implantação
2	Restrições	8	Conceitos transversais
3	Contexto e escopo	9	Decisões de arquitetura
4	Estratégia da solução	10	Requisitos de qualidade e testes
5	Visão de blocos de construção	11	Riscos e dívida técnica
6	Visão de tempo de execução	12	Glossário

Anexos: A — Links da entrega · B — Canvas de arquitetura · C — Evidências

1. Introdução e metas

O Mini CRM é um sistema para equipes pequenas de vendas: carteira de clientes, histórico de interações, funil de oportunidades e indicadores. Ele evolui o Mini CRM original (Azure Static Web Apps + Functions) para uma **arquitetura distribuída** com microfrontends, API Gateway, BFF, microsserviços, eventos e serverless.

1.1 Requisitos principais

Requisito	Como é atendido
Microfrontend (React)	Shell (host) e dois remotes (MF Clientes, MF Oportunidades) com Module Federation, publicados separadamente na Vercel.
BFF em Node.js com `GET /aggregated-data`	BFF NestJS que junta clientes, funil e indicadores num único JSON.
Microsserviço 1 — MongoDB Atlas, CRUD + Swagger	MS Clientes (Java 21, Spring Boot), Swagger em `/docs`.
Microsserviço 2 — Azure SQL, CRUD + Swagger	MS Oportunidades (Java 21, Spring Boot, JPA + Flyway), Swagger em `/docs`.
Azure Function (HTTP ou mensagem)	Uma Function com os dois gatilhos: HTTP `/api/indicadores` e Service Bus `oportunidade-ganha`.
AWS API Gateway	REST API na frente do BFF, com API key, throttling e CORS.
Clean Architecture + Vertical Slice	Camadas `domain / application / infrastructure / api`, com um pacote por caso de uso na aplicação.
Testes unitários e de arquitetura	JUnit 5 + ArchUnit (Java) e Jest/Vitest + dependency-cruiser (Node/React): 154 testes automatizados.
Imagens no Docker Hub e repositórios públicos	Cada serviço tem repositório próprio, CI e imagem `leandrocanha/minicrm-<serviço>`.

1.2 Metas de qualidade

Meta	Descrição
Manutenibilidade	Regras de dependência entre camadas e entre slices verificadas automaticamente (ArchUnit e dependency-cruiser).
Resiliência	Eventos com retentativa, dead-letter e idempotência; o dashboard tolera um serviço fora do ar.
Segurança	Front-end só fala com o Gateway (API key + throttling); segredos fora do código e dos repositórios.
Evolutividade	Serviços independentes, cada um com banco, repositório, pipeline e imagem próprios.

1.3 Partes interessadas

Papel	Expectativa
Vendedor	Ver e gerir apenas a própria carteira e o próprio funil.
Gerente de vendas	Ver a própria carteira e a dos vendedores da equipe.
Administrador	Ver tudo, inclusive o ranking de vendedores.
Professor / banca	Arquitetura aderente ao enunciado, evidências e demonstração só com URLs da nuvem.

2. Restrições

Restrição	Detalhe
Técnicas	O front nunca chama um microsserviço direto. Microsserviços não se chamam entre si: integram-se por evento. Imagens publicadas no Docker Hub. Demonstração só com URLs da nuvem.
Custos	Camadas gratuitas ou de estudante: MongoDB Atlas M0, Azure for Students, AWS com plano gratuito, Vercel Hobby.
Região da Azure	A política da assinatura de estudante só permite `centralus`, `spaincentral`, `northcentralus`, `francecentral` e `canadacentral`. Usamos `centralus`.
Organizacionais	Quatro alunos divididos entre a dupla A e o aluno B, um repositório por serviço, prazo final em 29/11/2026.
De negócio	Regras herdadas da SPEC do Mini CRM (RF03 a RF33), em especial RF15 (escopo por perfil) e RF30 (cliente vira Ativo ao ganhar oportunidade).
Linguagens	Back-end em Java 21 (decisão da dupla), BFF em Node.js (exigência), front em React 19 + Vite.

3. Contexto e escopo

3.1 Contexto de negócio

Três perfis de usuário (Vendedor, Gerente e Administrador) acessam o sistema pelo navegador. Não há integração com sistemas externos além dos serviços de nuvem listados no contexto técnico.

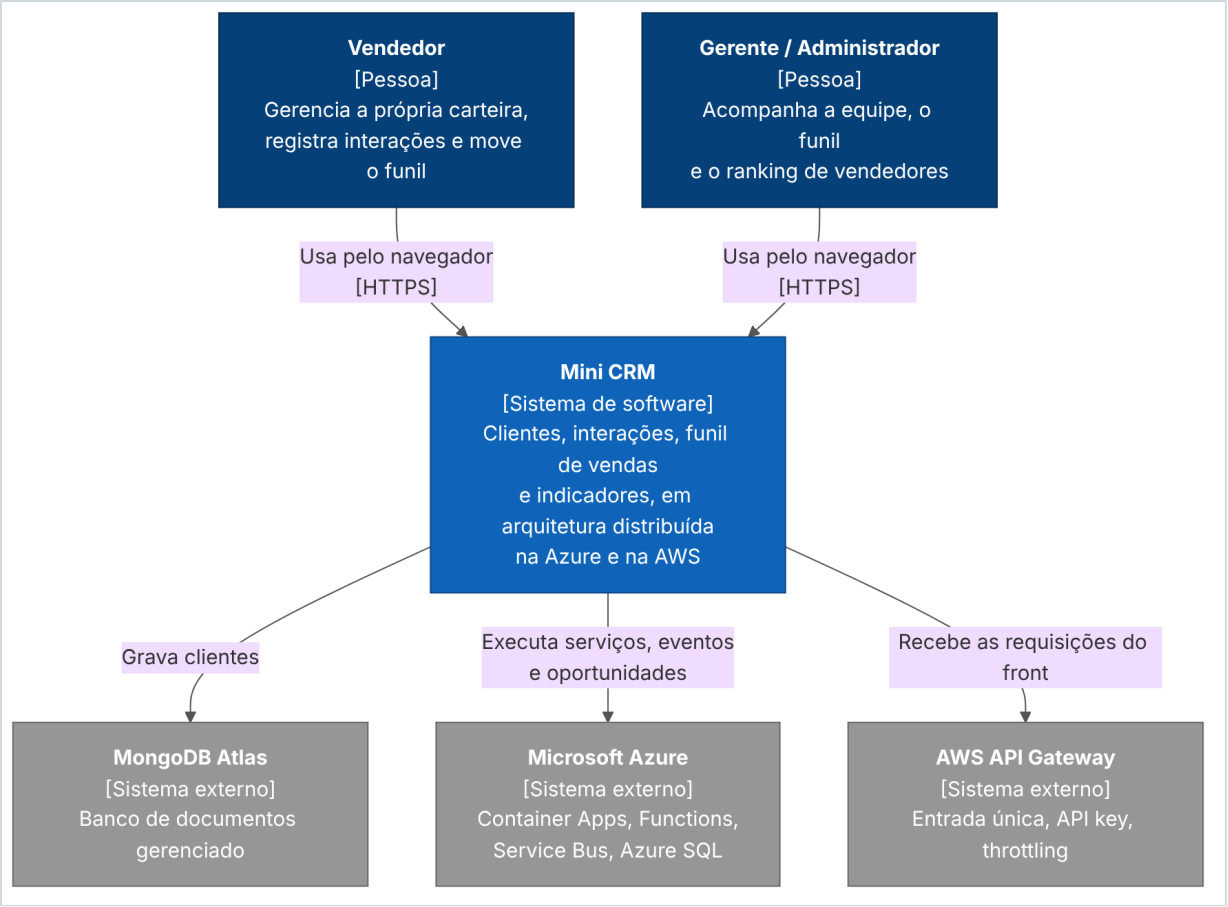


Figura 1 — C4 nível 1: contexto do sistema.

3.2 Contexto técnico

Interface	Protocolo e contrato
Usuário → Shell	HTTPS, Vercel.
Front → Gateway	HTTPS + header `x-api-key`; rotas `/clientes/*`, `/oportunidades/*`, `/aggregated-data`, sempre com `?usuarioId=`.
Gateway → BFF	HTTP proxy (`/{proxy+}`), timeout de 29 s.
BFF → MS Clientes / MS Oportunidades	REST, Swagger em `/docs`; `responsavelId` aceita lista para o escopo do RF15.
BFF → Function	`POST /api/indicadores` com `{clientes, oportunidades}` e header `x-functions-key` (OpenAPI).
MS Oportunidades → Function	Evento `OportunidadeGanha` na fila Service Bus `oportunidade-ganha` (AsyncAPI).
Function → MS Clientes	`POST /clientes/{id}/ativacao` com `{eventoId, oportunidadeId}`.

4. Estratégia da solução

Objetivo	Abordagem
Um ponto de entrada	Todo o tráfego do front passa pelo AWS API Gateway, que aplica API key, throttling (10 req/s, rajada de 20, 5.000 por dia) e CORS, e só então chega ao BFF.
Front simples para o domínio	O BFF calcula o escopo do usuário (RF15) e agrega as fontes; o front só pede `/aggregated-data`.
Domínios isolados	Um banco por domínio: MongoDB para Clientes, Azure SQL para Oportunidades. Cada serviço evolui e é implantado sozinho.
Integração desacoplada	Ganhar uma oportunidade publica um evento; a Function consome e aciona o MS Clientes. Nenhum microsserviço chama o outro.
Código sustentável	Clean Architecture com Vertical Slice, verificada por testes de arquitetura que quebram o build se a regra for violada.
Entrega contínua	Cada repositório tem pipeline de testes a cada push e publicação da imagem Docker numa tag `v*`.

5. Visão de blocos de construção

5.1 Nível 1 — containers

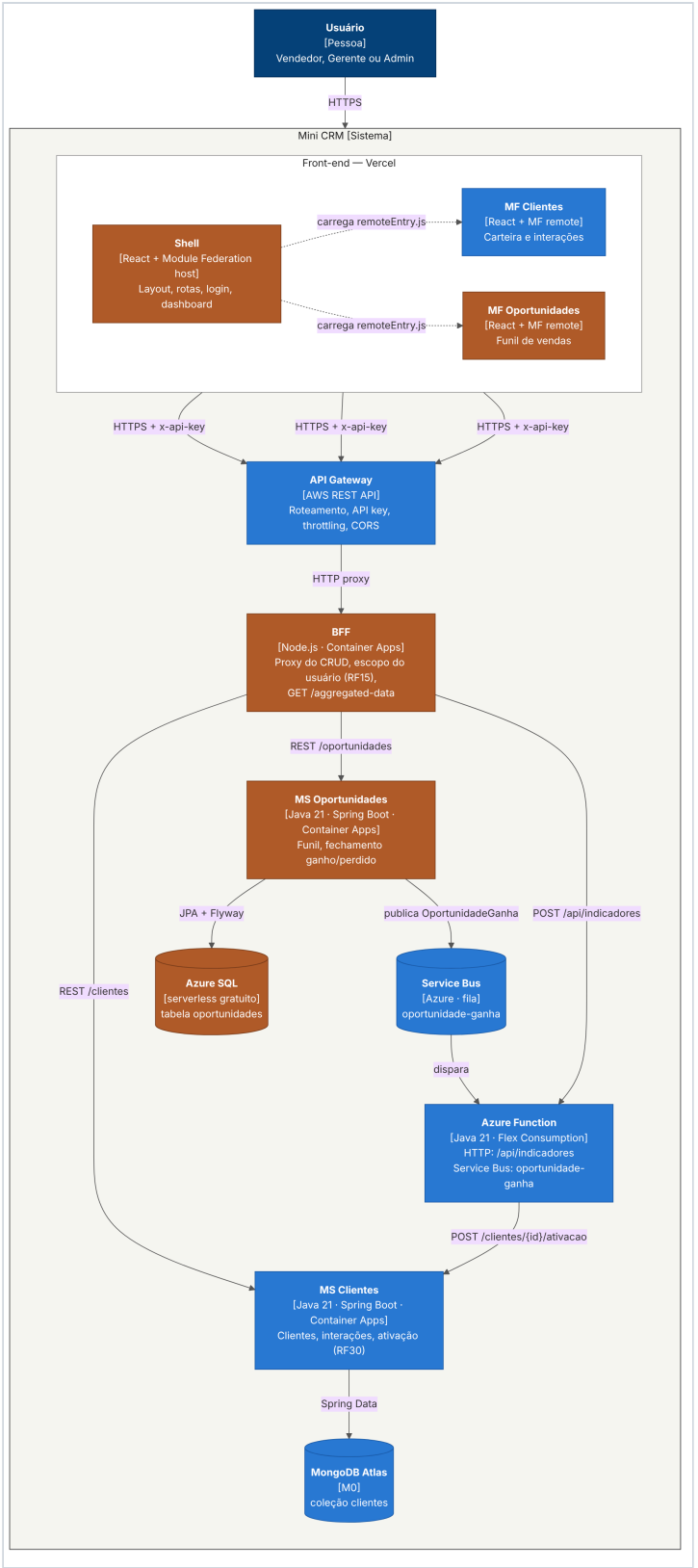


Figura 2 — C4 nível 2: containers. Azul: dupla A · laranja: aluno B.

5.2 Responsabilidades e repositórios

Bloco	Tecnologia	Dono	Repositório
Shell (host)	React 19, Vite, Module Federation	B	minicrm-arquitetura/shell
MF Clientes	React + remote	A	minicrm-arquitetura/mf-clientes
MF Oportunidades	React + remote	B	minicrm-arquitetura/mf-oportunidades
API Gateway	AWS REST API	A	minicrm-arquitetura/infra/aws
BFF	Node.js, NestJS	B	minicrm-bff
MS Clientes	Java 21, Spring Boot, MongoDB	A	minicrm-ms-clientes
MS Oportunidades	Java 21, Spring Boot, Azure SQL	B	minicrm-ms-oportunidades
Azure Function	Java 21, Flex Consumption	A	minicrm-function

5.3 Nível 2 — componentes internos

Os microsserviços Java seguem a mesma estrutura: `domain` (Java puro), `application` (um pacote por caso de uso, com `Command` e `Handler`), `infrastructure` (adaptadores de banco e mensageria), `api` (um controller por caso de uso, DTOs e Swagger) e `config` (raiz de composição). O BFF segue o mesmo desenho em TypeScript.

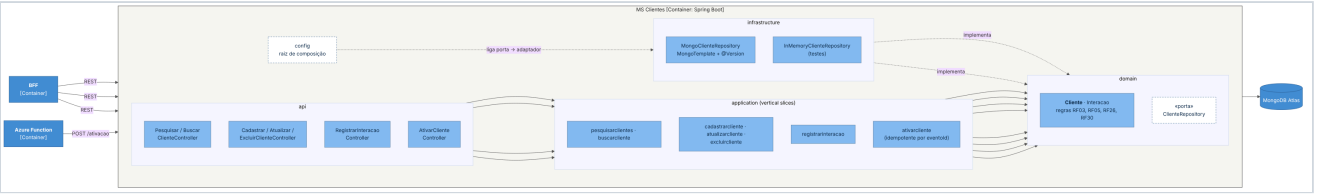


Figura 3 — C4 nível 3: componentes do MS Clientes (slices: cadastrar, buscar, pesquisar, atualizar, excluir, registrar interação, ativar cliente).

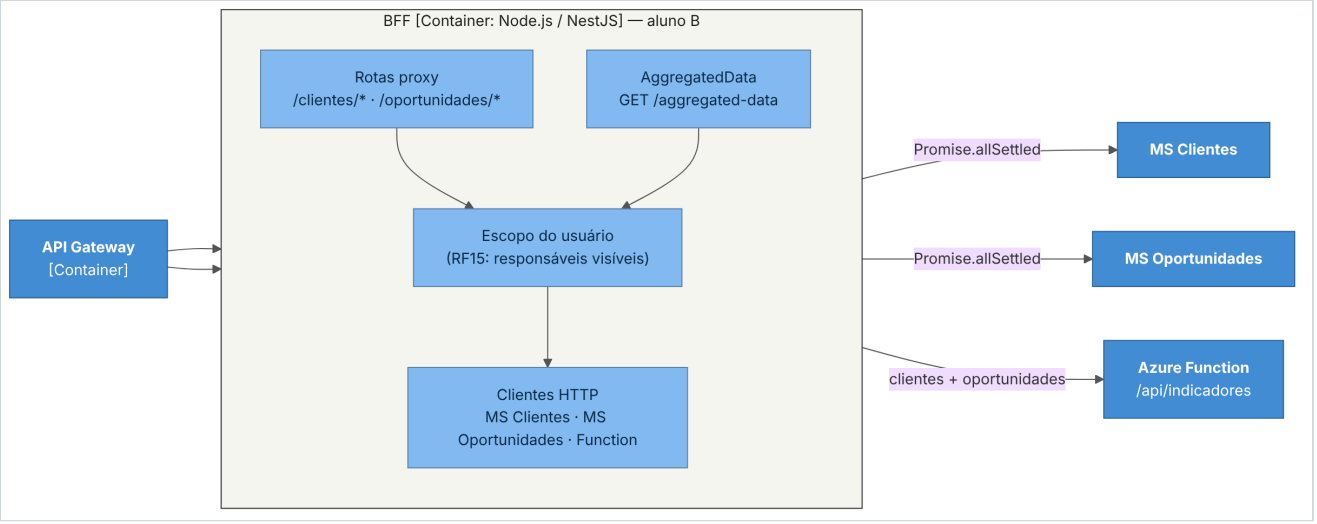


Figura 4 — C4 nível 3: componentes do BFF (slices: repasse de clientes, repasse de oportunidades, dados agregados).

Serviço	Slices (casos de uso)
MS Clientes	cadastrar, buscar, pesquisar, atualizar, excluir (lógica), registrar interação, ativar cliente (evento)

MS Oportunidades	criar, mover, listar funil, buscar, atualizar, excluir
Function	calcular indicadores (HTTP), ativar cliente (Service Bus)
BFF	repassar clientes, repassar oportunidades, obter dados agregados

6. Visão de tempo de execução

6.1 Cenário 1 — oportunidade ganha ativa o cliente (RF30)

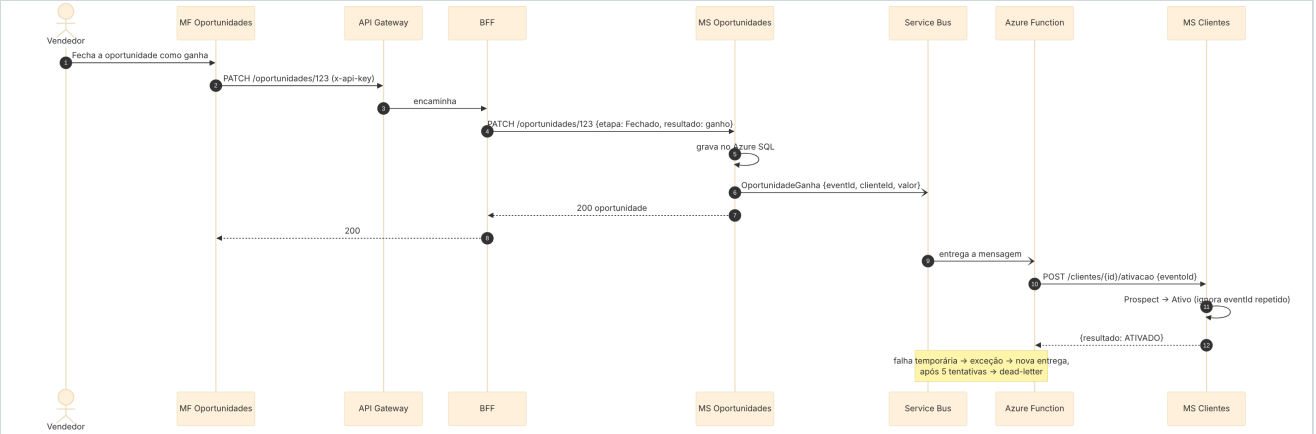


Figura 5 — Sequência do evento OportunidadeGanha.

1. No funil, o usuário fecha a oportunidade como **ganha**; a chamada passa pelo Gateway e pelo BFF até o MS Oportunidades.
2. O MS Oportunidades grava a mudança no Azure SQL e publica `OportunidadeGanha` (`eventoId`, `oportunidadeId`, `clienteId`, `valor`, `responsavelId`, `ocorridoEm`) na fila `oportunidade-ganha`.
3. A Function é disparada pela mensagem e chama `POST /clientes/{id}/ativacao` no MS Clientes.
4. O MS Clientes passa o cliente de Prospect para **Ativo** e responde com o resultado.

Idempotência. O MS Clientes guarda o `eventoId` já aplicado. Resultados possíveis: `ATIVADO`, `REPETIDO` (mesmo evento de novo, nada muda), `JA_ATIVO` e `IGNORADO_EXCLUIDO` (cliente excluído não volta a Ativo).

Resposta do MS Clientes	Tratamento na Function
200 (ATIVADO, JA_ATIVO, REPETIDO, IGNORADO_EXCLUIDO)	Mensagem concluída.
404	Pendência registrada no log; mensagem concluída.
Rede, timeout, 5xx, 409 CONCORRENCIA	Exceção: o Service Bus entrega de novo; depois de 5 tentativas (`maxDeliveryCount`) vai para dead-letter.
Mensagem malformada	Erro no log; mensagem concluída, pois repetir não adianta.

6.2 Cenário 2 — dashboard com `GET /aggregated-data`

1. O Shell chama `/aggregated-data?usuarioId=…` pelo Gateway.
2. O BFF calcula o escopo do usuário e busca, em paralelo, clientes (MS Clientes) e funil (MS Oportunidades).
3. Envia os dois resultados a `POST /api/indicadores` na Function, que devolve os indicadores (clientes por status, funil por etapa, valor em negociação, valor ganho, taxa de conversão, motivos de perda e ranking).

4. O BFF responde um único JSON: `{usuario, escopo, clientes, funil, indicadores, falhas[]}`.

A chamada usa `Promise.allSettled`: se um serviço estiver fora do ar ou lento (timeout configurável), ele aparece em `falhas[]` e o restante é devolvido, sem erro 500.

6.3 Cenário 3 — escopo por perfil (RF15)

Toda rota exige `?usuarioId=`. Sem ele: `400`; usuário desconhecido: `403`. O Administrador enxerga todos os responsáveis; o Gerente, a si e aos vendedores dele; o Vendedor, só a si. O BFF manda essa lista em `responsavelId=`, e pedir um responsável fora do escopo resulta em `403 FORA_DO_ESCOPO`.

7. Visão de implantação

Recurso	Serviço de nuvem	Configuração
Shell, MF Clientes, MF Oportunidades	Vercel (Hobby)	3 projetos do mesmo repositório, *Root Directory* diferente; variáveis `VITE_API_URL`, `VITE_API_KEY`, `VITE_MF_*_URL`.
API Gateway	AWS API Gateway REST, `us-east-1`	Stage `prod`, rota `/{proxy+}` para o BFF, API key com plano de uso (10 req/s, rajada 20, 5.000/dia), CORS.
BFF	Azure Container Apps, `centralus`	Imagem `leandrocanha/minicrm-bff:v1.0.0`, porta 3000, 1 a 2 réplicas, segredo da chave da Function.
MS Clientes	Azure Container Apps, `centralus`	Imagem `leandrocanha/minicrm-ms-clientes`, porta 8080, segredo da conexão com o MongoDB.
MS Oportunidades	Azure Container Apps, `centralus`	Imagem `leandrocanha/minicrm-ms-oportunidades:v1.0.0`, porta 8080, segredos do SQL e do Service Bus.
Function	Azure Functions (Flex Consumption, Java 21)	Gatilhos HTTP e Service Bus; configurações `ServiceBusConnection` e `MS_CLIENTES_URL`.
Fila	Azure Service Bus (Basic)	Fila `oportunidade-ganha`, política *Send* (MS Oportunidades) e *Listen* (Function).
Banco de Clientes	MongoDB Atlas (M0)	Coleção de clientes e interações.
Banco de Oportunidades	Azure SQL (Basic)	Tabela `oportunidades`, criada por migração Flyway.
Imagens	Docker Hub	`leandrocanha/minicrm-ms-clientes`, `…-ms-oportunidades`, `…-bff`.

Os recursos da Azure ficam no grupo `rg-minicrm`, no ambiente `cae-minicrm`. A criação foi automatizada nos scripts `infra/azure/deploy-pessoa-a.sh` e `deploy-pessoa-b.sh`, e a do gateway em `infra/aws/criar-gateway.sh`.

8. Conceitos transversais

Conceito	Como é tratado

Clean Architecture	Dependências apontam para dentro: `api` e `infrastructure` dependem de `application` e `domain`, nunca o contrário. `domain` e `application` são Java puro (sem Spring) no back-end.
Vertical Slice	Cada caso de uso é um pacote com seu Command e Handler. Uma slice não depende de outra, apenas de `application.common`.
Raiz de composição	`config` (Java) e `app.module.ts` (BFF) ligam portas e adaptadores; só eles conhecem tudo.
Concorrência	Concorrência otimista (`@Version`) nos dois serviços, com até 3 retentativas no MS Clientes.
Idempotência	Eventos identificados por `eventoId`; reaplicar não muda o estado.
Erros	Formato único `{status, codigo, erro, erros[]}`, repassado sem alteração pelo BFF para o front mostrar erros campo a campo.
Segurança	API key e throttling no Gateway; segredos (conexões de banco, Service Bus, chave da Function) em segredos do Container Apps e do GitHub Actions, nunca no código; CORS restrito ao Shell no BFF.
Exclusão lógica	Cliente excluído vira Inativo e some das listagens, mas o histórico é preservado (RF03).
Contratos	OpenAPI (Swagger) nos três serviços HTTP e AsyncAPI para o evento.
Observabilidade	Logs dos Container Apps no Log Analytics; resumo dos testes publicado como check do commit no GitHub.

9. Decisões de arquitetura

#	Decisão	Motivo e consequência
1	Java 21 + Spring Boot nos microsserviços	Domínio da equipe; ArchUnit para verificar a arquitetura. BFF em Node.js, como o enunciado pede.
2	Dois bancos, um por domínio (MongoDB × Azure SQL)	Atende o enunciado e isola os domínios. Custo: não há joins entre clientes e oportunidades, o BFF agrega.
3	Evento `OportunidadeGanha` em vez de chamada direta	Microsserviços desacoplados; retentativa e dead-letter do Service Bus. Custo: consistência eventual (o cliente vira Ativo segundos depois).
4	Uma Function com dois gatilhos	Mostra HTTP e mensagem com um único deploy. Os indicadores são cálculo puro, adequado a serverless.
5	BFF calcula o escopo (RF15)	O front não decide permissões e os microsserviços recebem só filtros prontos.
6	Gateway da AWS com `/{proxy+}`	Uma definição curta, sem repetir cada rota; CORS e chave num só lugar.
7	Um repositório por serviço + um central	Pipeline, imagem e versão independentes por serviço; o central guarda frontends, docs e infra.

8	Imagens `leandrocanha/minicrm-<serviço>`	O Docker Hub só aceita `usuário/repositório`, por isso hífen em vez de `usuário/minicrm/serviço`.
9	Região `centralus` na Azure	Política da assinatura de estudante bloqueou `brazilsouth`.
10	Concorrência otimista com `@Version`	Evita bloqueios e perda de atualização sem custo de lock.

10. Requisitos de qualidade e testes

10.1 Cenários de qualidade

Atributo	Cenário	Resposta esperada
Manutenibilidade	Alguém importa uma classe do Spring em `domain`, ou uma slice usa outra.	O ArchUnit falha o build do serviço.
Resiliência	O MS Oportunidades fica fora do ar durante o dashboard.	`/aggregated-data` responde com os clientes e lista o serviço em `falhas[]`.
Resiliência	A Function recebe o mesmo evento duas vezes.	Resultado `REPETIDO`; o estado do cliente não muda.
Segurança	Requisição sem `x-api-key`.	O Gateway responde `403` sem chegar ao BFF.
Segurança	Vendedor pede clientes de outro vendedor.	`403 FORA_DO_ESCOPO`.
Desempenho	Pico de requisições.	Throttling do plano de uso (10 req/s, 5.000/dia).

10.2 Testes automatizados (CI verde)

Componente	Testes	Tipos
MS Clientes	33	Unitários de domínio e casos de uso, API, ArchUnit
Azure Function	29	Unitários (indicadores, evento), gateway HTTP, ArchUnit
MS Oportunidades	25	Unitários de domínio, API com H2, ArchUnit
BFF	37	Unitários, API (supertest), arquitetura (dependency-cruiser)
MF Clientes	9	Componentes e cliente de API (Vitest)
MF Oportunidades	13	Componentes e cliente de API (Vitest)
Shell	8	Dashboard, seletor de usuário, isolamento de falha dos MFs
Total	**154**	Todos passam no GitHub Actions de cada repositório.

11. Riscos e dívida técnica

Risco	Mitigação / situação
Cold start dos Container Apps e do SQL na demonstração	BFF e MS Oportunidades mantêm uma réplica; aquecer os demais serviços antes de gravar o vídeo.
Evento perdido se o Service Bus falhar depois de gravar no SQL	Dívida técnica: padrão **Outbox** (gravar o evento na mesma transação e publicar depois).
Limite de 29 s do API Gateway	Timeout do BFF em 20 s, abaixo do limite.
Chave da API no código do front	É pública por natureza; serve para limitar uso, não como segredo. Um login real (token) seria a evolução natural.
Usuários simulados	O seletor “Entrar como” usa a massa de usuários do Mini CRM; não há autenticação real.
Créditos de nuvem expiram	Usar apenas durante a avaliação e apagar os recursos depois (`az group delete -n rg-minicrm` e remoção do gateway).

12. Glossário

Termo	Significado
BFF	Backend for Frontend: camada de API feita sob medida para o front.
Microfrontend (MF)	Parte do front publicada e versionada separadamente e carregada pelo Shell.
Module Federation	Mecanismo que deixa o Shell carregar o `remoteEntry.js` de cada MF em tempo de execução.
Vertical Slice	Organizar o código por caso de uso, não por camada técnica.
ArchUnit / dependency-cruiser	Ferramentas que testam regras de dependência entre pacotes.
Dead-letter	Fila de mensagens que esgotaram as tentativas de entrega.
Idempotência	Aplicar a mesma operação mais de uma vez tem o mesmo efeito que aplicar uma só.
RF15 / RF30	Requisitos do Mini CRM: escopo por perfil e “cliente vira Ativo ao ganhar oportunidade”.

Anexo A — Links da entrega

Item	URL
GitHub — repositório central	https://github.com/Le3Dev/minicrm-arquitetura
GitHub — MS Clientes	https://github.com/Le3Dev/minicrm-ms-clientes
GitHub — Function	https://github.com/Le3Dev/minicrm-function
GitHub — MS Oportunidades	https://github.com/Le3Dev/minicrm-ms-oportunidades
GitHub — BFF	https://github.com/Le3Dev/minicrm-bff
Docker Hub	https://hub.docker.com/u/leandrocanha
Front (Shell)	https://shell-zeta-puce.vercel.app
MF Clientes	https://minicrm-arquitetura.vercel.app
MF Oportunidades	https://minicrm-arquitetura-1xxh.vercel.app
API Gateway (exige `x-api-key`)	https://4sbv1bj2qj.execute-api.us-east-1.amazonaws.com/prod
Swagger do BFF	https://bff.kindbay-40f0b2c9.centralus.azurecontainerapps.io/docs
Swagger do MS Clientes	https://ms-clientes.kindbay-40f0b2c9.centralus.azurecontainerapps.io/docs
Swagger do MS Oportunidades	https://ms-oportunidades.kindbay-40f0b2c9.centralus.azurecontainerapps.io/docs
Swagger da Function (HTTP)	https://func-minicrm-10059.azurewebsites.net/api/docs
Eventos da Function (AsyncAPI)	https://func-minicrm-10059.azurewebsites.net/api/eventos
Vídeo (YouTube, não listado)	https://youtu.be/f_G2twBHaKU

Anexo B — Canvas de arquitetura



Canvas de arquitetura do Mini CRM distribuído.

Anexo C — Evidências

C.1 Aplicação no ar

Tela de Clientes do Mini CRM em shell-zeta-puce.vercel.app/clientes, carregada pelo caminho completo: Shell (Vercel) → AWS API Gateway → BFF → MS Clientes → MongoDB Atlas.

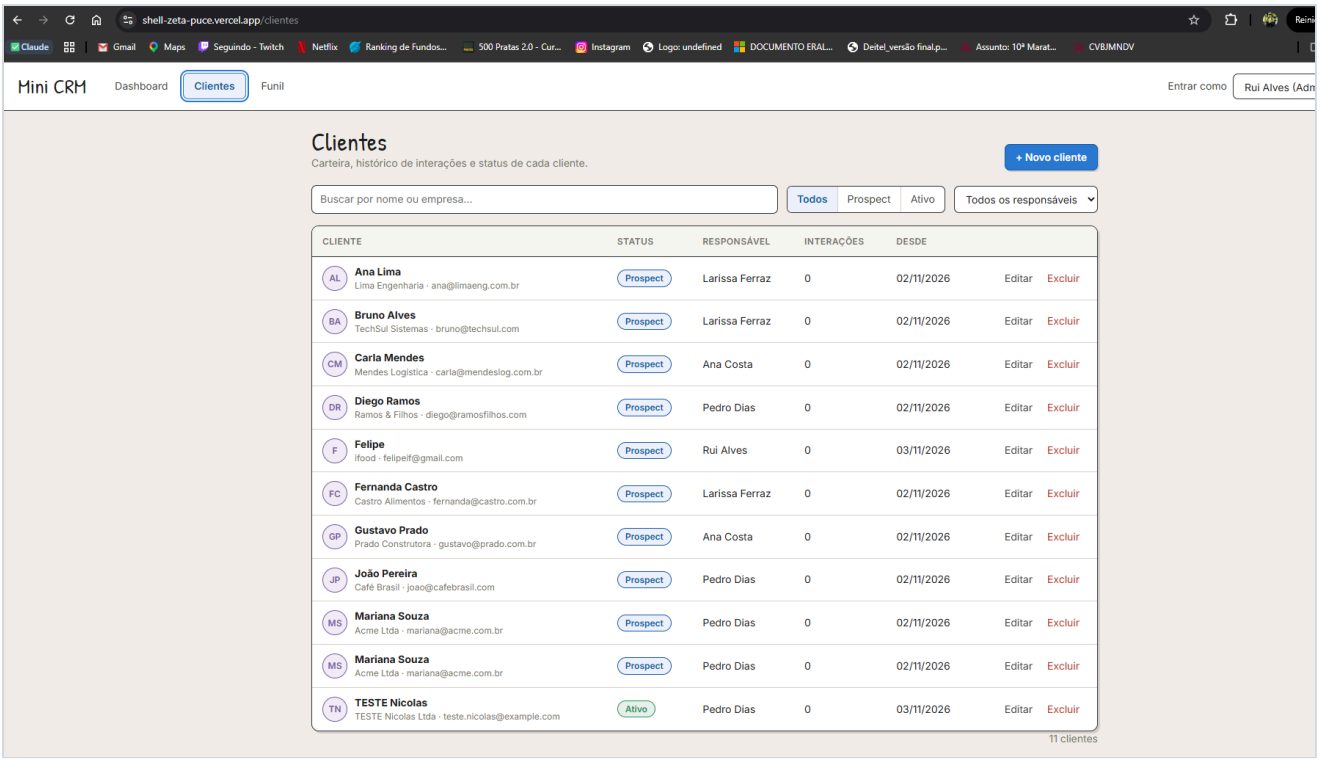


Figura C.1 — MF Clientes dentro do Shell, com os dados vindos do MongoDB Atlas pela nuvem.

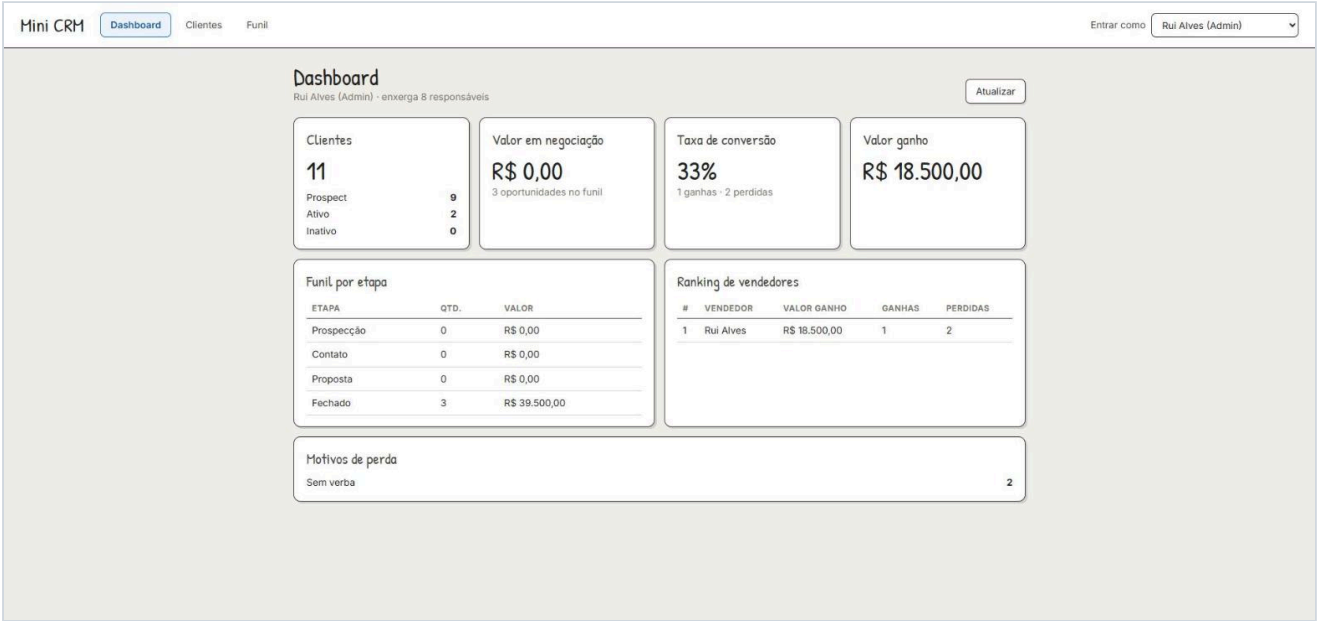


Figura C.2 — Dashboard do Shell (Vercel): dados de `GET /aggregated-data` (clientes do MongoDB, funil do Azure SQL e indicadores da Azure Function).

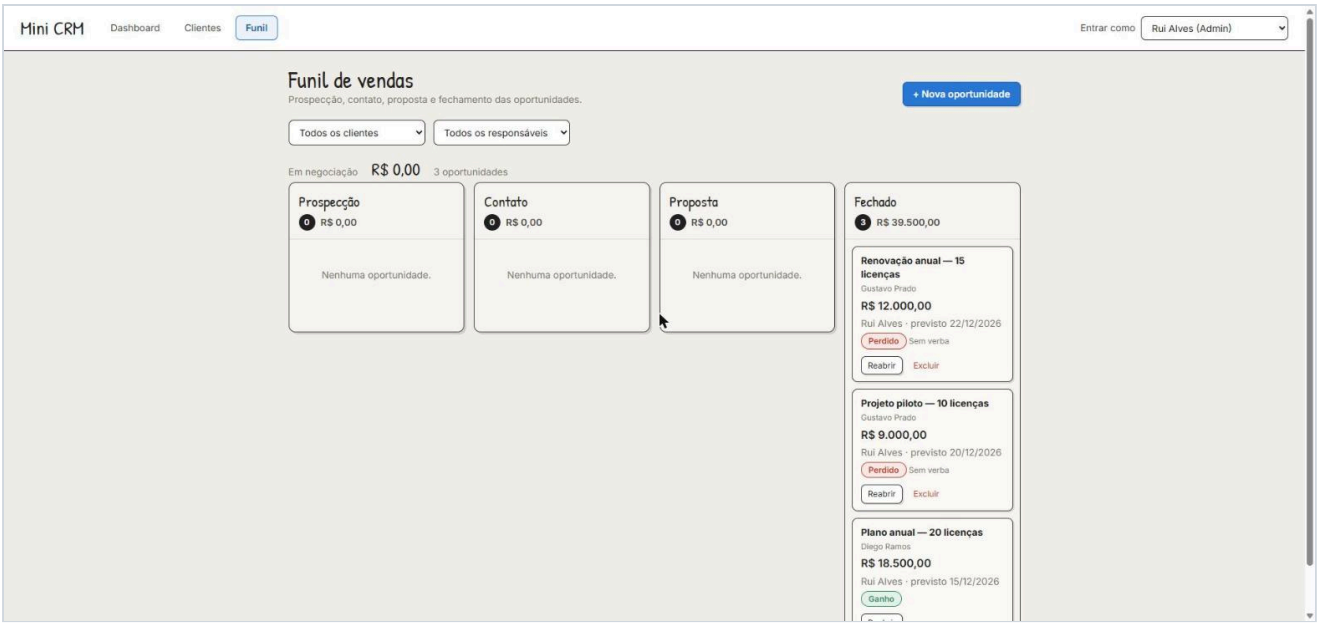


Figura C.3 — Funil de vendas (MF Oportunidades): oportunidades perdidas com motivo e oportunidade ganha.

C.2 Swagger do BFF

bff.kindbay-40f0b2c9.centralus.azurecontainerapps.io/docs

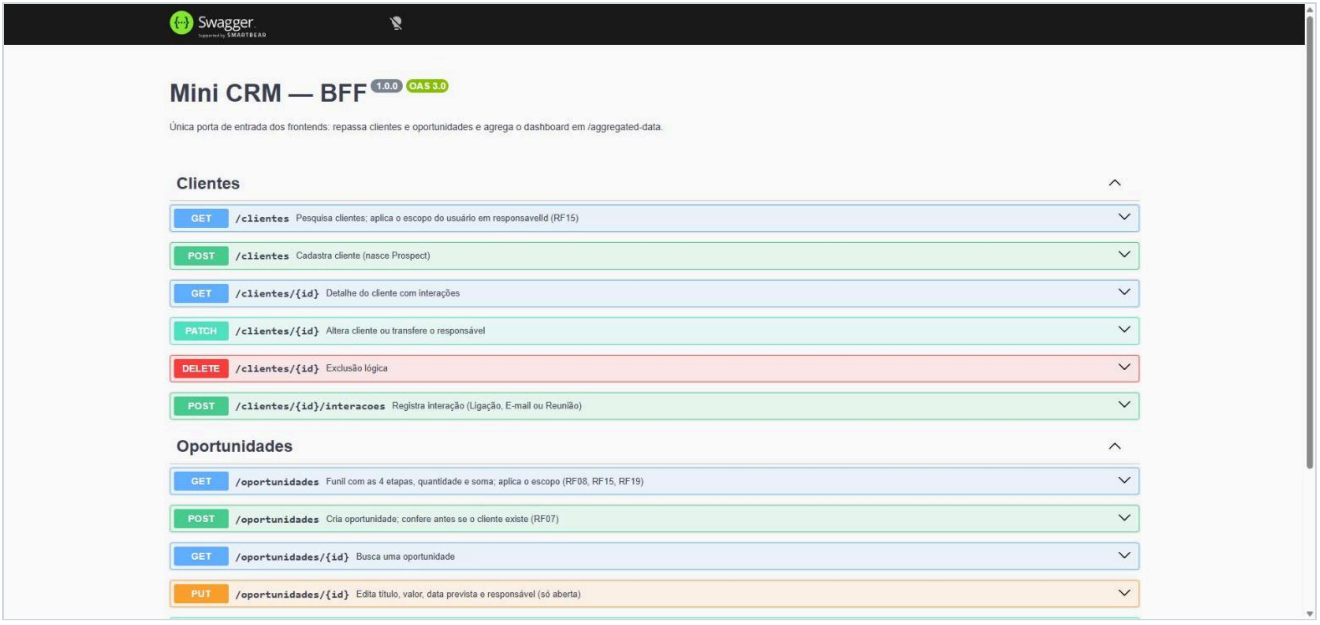


Figura C.4 — Swagger do BFF (Azure Container Apps).

C.3 Swagger dos microsserviços

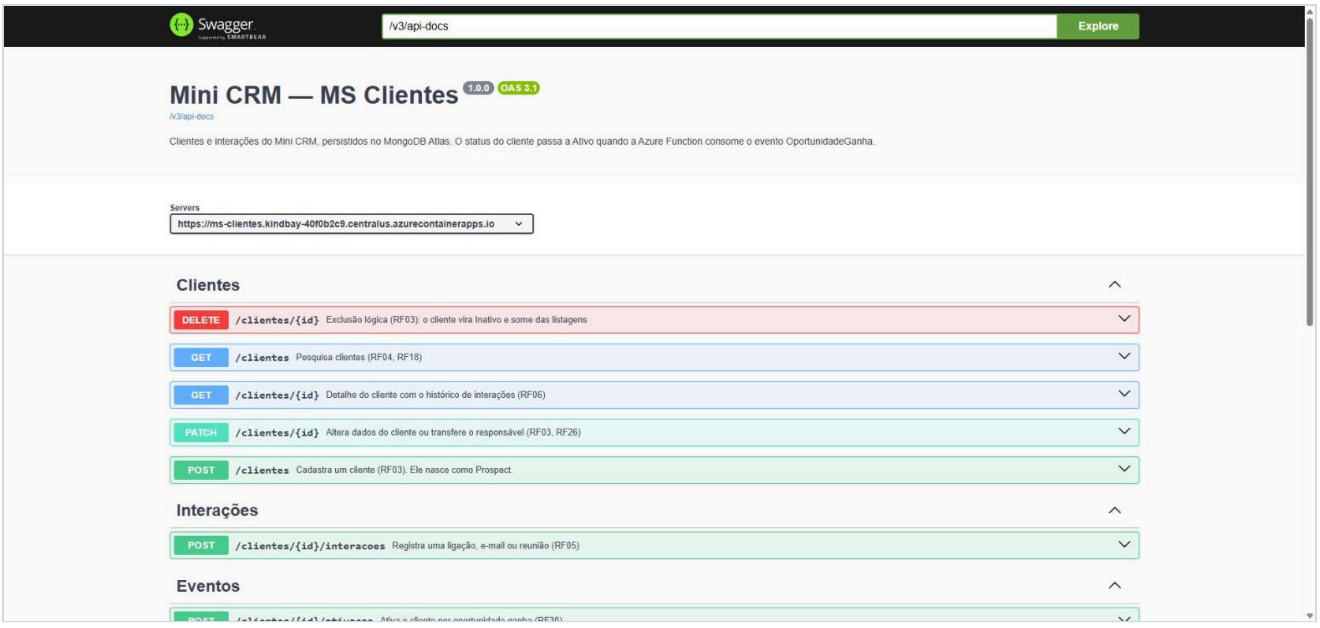


Figura C.5 — Swagger do MS Clientes (MongoDB Atlas), com o endpoint de eventos de ativação.

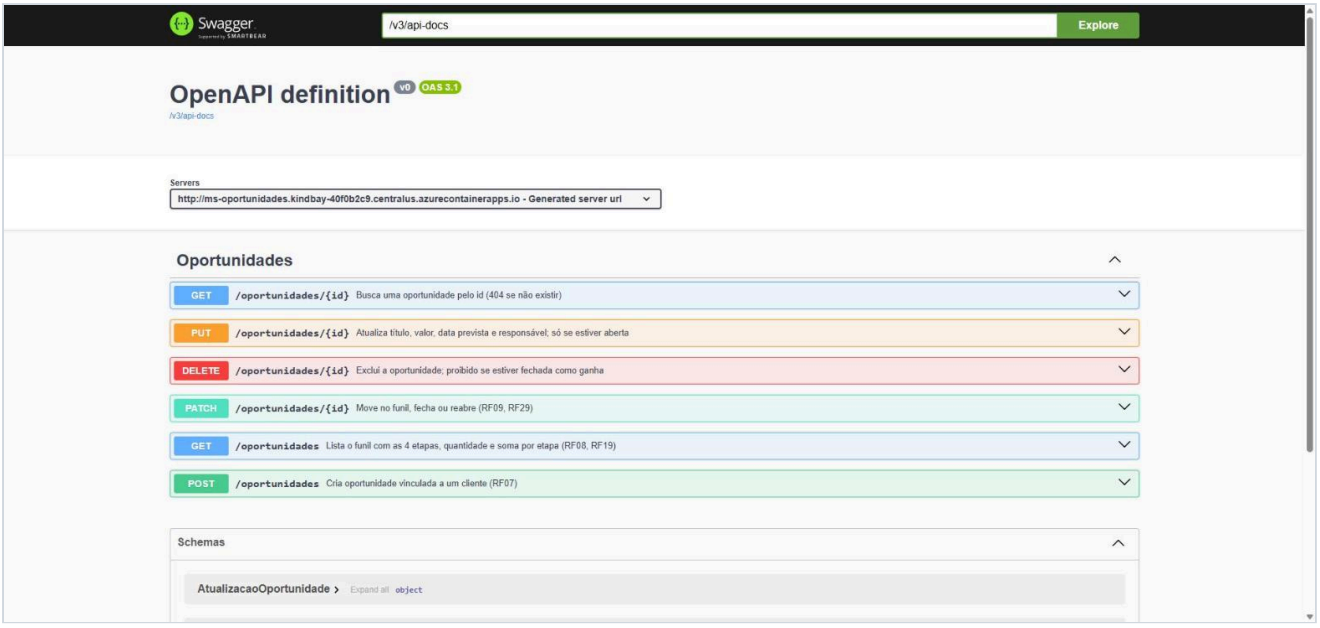


Figura C.6 — Swagger do MS Oportunidades (Azure SQL).

C.4 Azure Function e eventos

A Function expõe `POST /api/indicadores` (HTTP) e consome o evento `OportunidadeGanha` do Service Bus (contrato em `/api/eventos`).

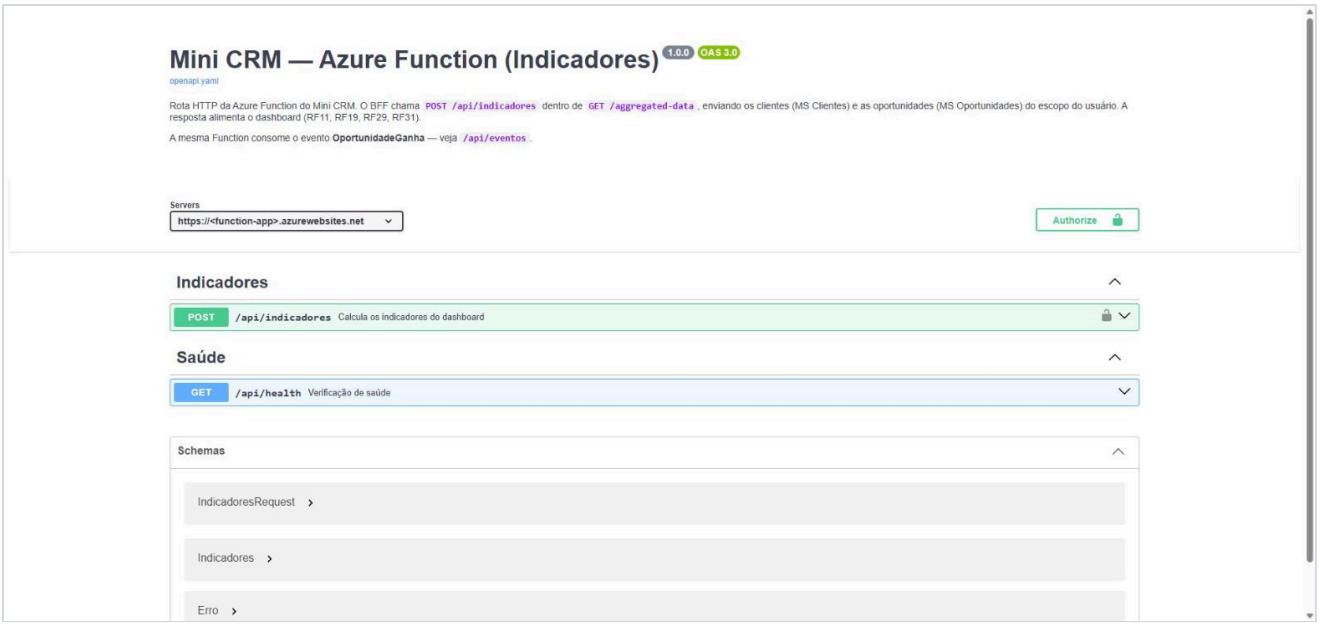


Figura C.7 — Documentação da Azure Function: func-minicrm-10059.azurewebsites.net/api/docs.

C.5 API Gateway (AWS)

O Gateway exige a chave de acesso (API key). Com a chave, a rota `GET /aggregated-data` responde 200 com o JSON agregado pelo BFF; sem a chave, o Gateway recusa com 403.

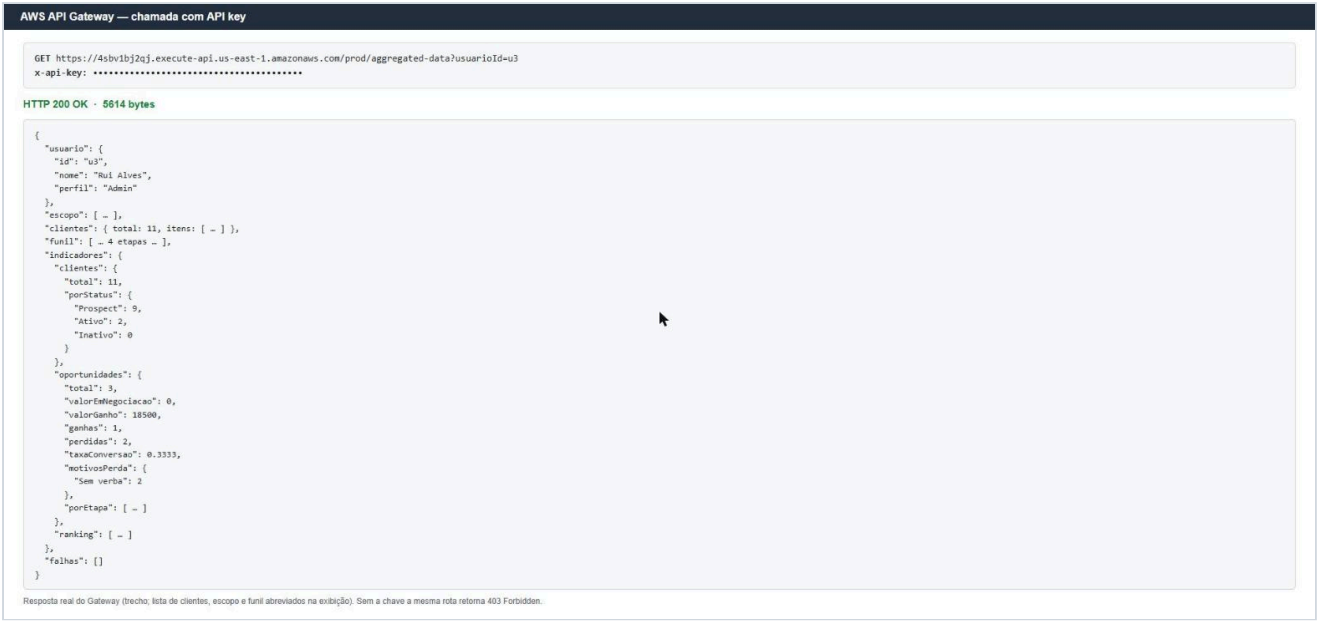


Figura C.8 — Chamada ao API Gateway com a chave no header `x-api-key` (valor oculto): HTTP 200 e resposta do BFF (trecho; listas abreviadas na exibição).

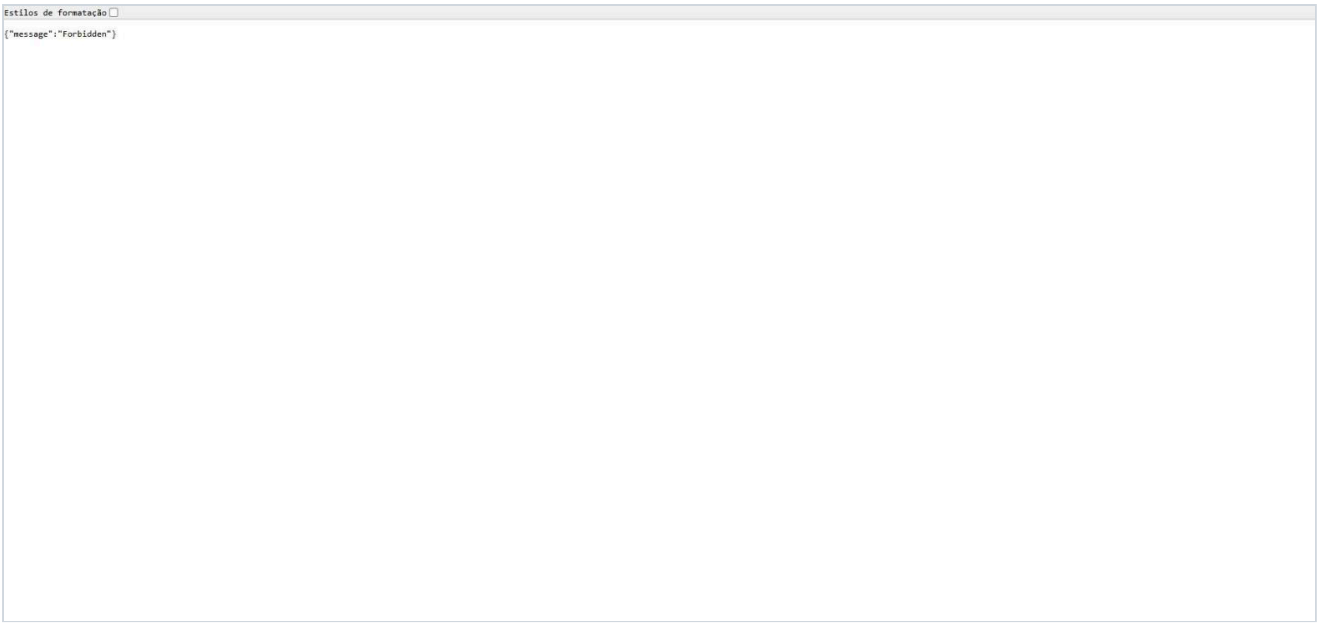


Figura C.9 — A mesma rota sem a chave: `{"message":"Forbidden"}` (403).